Panduan Ujian Chunin

Cloud-Native Fullstack Application & Automated CI/CD Pipeline

Mata Kuliah:	Cloud Computing	**Bentuk Tugas:**	Proyek Kelompok (3–4 Mahasiswa)
Dosen Pengampu:	CSO	**Sifat Pengumpulan:**	Wajib (Bobot Nilai UTS)

1. Deskripsi Umum Tugas

Tugas Kelompok UTS ini dirancang untuk menguji pemahaman mahasiswa secara praktis dalam membangun, mengisolasi, dan mengotomatisasi penyebaran (*deployment*) aplikasi web fullstack sederhana menggunakan **Docker**, **Docker Named Volumes**, **Custom Bridge Network**, **Docker Compose**, dan **Jenkins CI/CD Pipeline** terintegrasi dengan **GitHub Webhook**. Setiap kelompok (terdiri dari **3–4 mahasiswa**) wajib memilih **satu dari 10 pilihan studi kasus** yang disediakan. Sisi pemrograman (*coding*) sengaja dibuat sederhana agar mahasiswa dapat berfokus 100% pada penerapan arsitektur *containerization* dan otomatisasi *DevOps*.

2. Spesifikasi Arsitektur Teknis Minimum

1. **Backend Service (Python REST API):** Dibangun dengan Python (app.py) tanpa framework berat. Memiliki fitur REST API berbasis JSON (GET /messages, POST /messages, dan DELETE /messages?index=X). Seluruh data wajib disimpan ke file JSON di dalam **Docker Named Volume** (misal: -v backend-data:/var/lib/backend_data).
2. **Frontend Service (Plain HTML & Vanilla JS):** Dibuat dengan HTML & Vanilla JS biasa (index.html) tanpa framework. Menggunakan fetch() API untuk komunikasi asynchronous dengan Backend. Dijalankan di web server **Nginx** menggunakan **Bind Mount** ke folder host lokal agar perubahan UI langsung terlihat tanpa rebuild image.
3. **Infrastruktur & Isolation (Docker & Networking):** Kontainer Frontend dan Backend terhubung dalam **Custom Bridge Network** (app-net) terisolasi dan berkomunikasi via Service Name (DNS Docker). Seluruh stack didefinisikan secara deklaratif di file docker-compose.yml.
4. **Otomatisasi CI/CD (Jenkins & GitHub):** Seluruh kode disimpan di GitHub. Setiap git push ke branch main otomatis memicu Jenkins via **GitHub Webhook**. Skrip **Declarative Jenkinsfile** memiliki 5 Stage utama (*Checkout, Syntax Check, Build Image, Automated Deployment, Health Check*).

3. 10 Pilihan Studi Kasus Kelompok (Pilih Salah Satu)

Setiap kelompok wajib memilih salah satu studi kasus di bawah ini untuk diimplementasikan:

1. Buku Tamu Digital Kampus (Digital Guestbook)
• **Skenario:** Sistem pencatat daftar tamu/pengunjung acara kampus.
• **Fitur API & UI:** GET /messages (Daftar tamu), POST /messages (Form tamu baru), DELETE /messages?index=X (Hapus entri tamu).
• **Storage Volume:** guestbook.json dalam Named Volume guestbook-db-data
2. Papan Pengumuman Kampus (Campus Announcement Board)
• **Skenario:** Portal informasi pengumuman perkuliahan & kegiatan Ormawa.
• **Fitur API & UI:** GET /announcements (Daftar pengumuman), POST /announcements (Tambah pengumuman), DELETE /announcements?index=X (Hapus pengumuman kadaluarsa).
• **Storage Volume:** announcements.json dalam Named Volume announcement-data
3. Antrean Layanan Mahasiswa (Student Service Queue System)
• **Skenario:** Aplikasi nomor antrean sederhana untuk layanan administrasi/LAA.
• **Fitur API & UI:** GET /queue (Daftar antrean), POST /queue (Ambil nomor antrean), DELETE /queue?index=0 (Panggil/selesai antrean).

• **Storage Volume:** queue.json dalam Named Volume queue-db-data
4. Inventaris Barang Laboratorium (Lab Inventory Catalog)
• **Skenario:** Sistem pencatatan peminjaman & ketersediaan alat lab (Mikrotik, Modem, Kabel).
• **Fitur API & UI:** GET /inventory (Katalog alat), POST /inventory (Tambah alat baru), DELETE /inventory?index=X (Hapus alat rusak/dipindah).
• **Storage Volume:** inventory.json dalam Named Volume lab-inventory-data
5. Log & Hit Counter Pengunjung Web (Web Visitor Counter Log)
• **Skenario:** Sistem pencatat statistik kunjungan situs web laboratorium.
• **Fitur API & UI:** GET /logs (Total hit & log kunjungan), POST /logs (Tambah log baru), DELETE /logs?index=X (Hapus log tertentu).
• **Storage Volume:** visitor_logs.json dalam Named Volume visitor-log-data
6. Papan Catatan Tugas Kelompok (Team Task & Note List)
• **Skenario:** Aplikasi todo list sederhana untuk membagi tugas antar-anggota tim.
• **Fitur API & UI:** GET /tasks (Daftar tugas), POST /tasks (Tambah tugas baru), DELETE /tasks?index=X (Hapus/selesai tugas).
• **Storage Volume:** tasks.json dalam Named Volume team-task-data
7. Sistem Polling / Voting Kampus (Campus Polling System)
• **Skenario:** Aplikasi pemungutan suara sederhana untuk kegiatan kampus.
• **Fitur API & UI:** GET /votes (Perolehan suara Opsi A vs B), POST /votes (Kirim suara), DELETE /votes?index=X (Reset catatan suara).
• **Storage Volume:** votes.json dalam Named Volume polling-db-data
8. Laporan Kerusakan Fasilitas Kampus (Facility Issue Reporting)
• **Skenario:** Form aduan mahasiswa untuk fasilitas kelas (AC mati, proyektor rusak).
• **Fitur API & UI:** GET /reports (Daftar aduan), POST /reports (Kirim aduan baru), DELETE /reports?index=X (Hapus aduan yang selesai).
• **Storage Volume:** facility_reports.json dalam Named Volume facility-data
9. Repositori Tautan Modul Praktikum (Resource Link Hub)
• **Skenario:** Portal agregator tautan modul, slide, dan materi praktikum.
• **Fitur API & UI:** GET /links (Daftar link materi), POST /links (Tambah link baru), DELETE /links?index=X (Hapus link non-aktif).
• **Storage Volume:** links.json dalam Named Volume link-hub-data
10. Portofolio & Profil Kelompok (Group Team Profile Showcase)
• **Skenario**: Halaman web profil anggota kelompok dan daftar keahlian.
• **Fitur API & UI:** GET /members (Daftar profil & skill), POST /members (Tambah anggota/skill), DELETE /members?index=X (Hapus entri).
• **Storage Volume:** team_members.json dalam Named Volume team-profile-data

4. Panduan Pembuatan GitHub Personal Access Token (PAT)

Untuk mengintegrasikan repositori GitHub dengan Jenkins secara aman, setiap kelompok wajib menggunakan **Personal Access Token (PAT)**:

1. Login ke akun **GitHub** → Klik foto profil di kanan atas → **Settings**.
2. Gulir ke bawah pada menu sebelah kiri → **Developer settings**.
3. Pilih **Personal access tokens** → **Tokens (classic)** → **Generate new token (classic)**.
4. Isi **Note**: Jenkins CI/CD UTS Token dan tentukan **Expiration**: 30 days atau 90 days.
5. Centang izin wajib: [x] **repo** (Akses repositori) & [x] **admin:repo_hook** (Webhook).
6. Klik **Generate token**, salin kode token (ghp_xxxx...), lalu tambahkan pada Jenkins di menu **Manage Jenkins** → **Credentials** → **Global** → **Add Credentials** (Kind: *Username with password*).

5. Rubrik Penilaian UTS (Skala Total 100%)

Komponen	Bobot	Sangat Baik (85–100)	Cukup (70–84)	Perlu Perbaikan (<70)
1. Backend Python API	20%	Fitur CRUD lengkap (GET, POST, DELETE), respon JSON rapi, error handling (400/404) presisi.	API berjalan tetapi hanya mendukung 2 metode (misal: GET & POST), error handling belum sempurna.	API sering crash, tidak menangani JSON dengan benar, atau hanya ada 1 endpoint GET.
2. Frontend HTML & JS	15%	Tampilan UI bersih, fetch() JS lancar, tombol Delete berfungsi real-time.	UI berfungsi, tetapi pembaruan data harus refresh manual atau async JS belum rapi.	UI tidak terhubung ke Backend API atau terdapat error JS di console.
3. Docker & Storage Config	25%	Named Volume terbukti menjaga data saat kontainer di-dismantle (docker rm -f), Bind Mount berfungsi.	Storage berfungsi tetapi salah menerapkan jenis volume (misal: DB pakai Bind Mount).	Data hilang saat kontainer di-restart, atau kontainer tanpa isolasi volume.
4. Jenkins CI/CD Pipeline	25%	Jenkinsfile sukses mengeksekusi seluruh stage (Checkout, Build, Deploy, Health Check) via Webhook.	Pipeline berhasil dijalankan manual (Build Now), namun Webhook GitHub belum otomatis.	Pipeline gagal di tengah jalan (red build) atau tidak memakai Jenkinsfile.
5. Dokumentasi & Demo	15%	README.md memuat diagram arsitektur, petunjuk lengkap, & video demo memperlihatkan git push hingga auto-deploy.	Dokumentasi ada tetapi kurang detail, atau video demo tidak memperlihatkan CI/CD utuh.	Tidak ada README.md atau tidak melampirkan video demo/presentasi kelompok.

6. Berkas Yang Wajib Dikumpulkan (Deliverables)

Setiap kelompok wajib mengumpulkan berkas-berkas berikut melalui LMS / Portal Kampus:

1. **Tautan Repositori GitHub**: Berisi app.py, index.html, Dockerfile, docker-compose.yml, dan Jenkinsfile.
2. **Berkas README.md**: Memuat Nama & NIM Anggota Kelompok, studi kasus yang dipilih, diagram arsitektur Docker, serta langkah instalasi lokal & Jenkins.
3. **Video Demo / Presentasi (Durasi 5–8 Menit)**: Memperlihatkan demonstrasi UI & API, simulasi Data Immunity (menghapus kontainer backend), serta simulasi CI/CD (melakukan git push hingga Jenkins ter-trigger otomatis).